



Angular Avançado

Sessão 3

Sub-componentes e design reativo com Signals

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficamos na Sessão 2

Recapitulação rápida antes de continuar

Acesso a dados

- HttpClient: imperativo, subscribe, todos os verbos
- httpResource: declarativo, entra signal / sai signal
- `value · error · isLoading · status`
- `undefined` desativa · race conditions tratadas

Template por estado

- Um @if por estado: dados / erro / loading
- @switch (status()) com @default never
- @let s = status() para narrowing
- type="button" para não fazer submit

Sub-componentes (início)

- Desenhar a API a partir do pai
- `<app-flight-card [item] [selected] (selectedChange)>`
- basket como Record<number, boolean>
- updateBasket com spread → nova referência

Hoje: implementar o FlightCard que desenhámos, e depois entrar no design reativo — computed, resources e effects.

Agenda de hoje

01 Sub-componentes (cont.)

input()/output() · @let · content projection · model() · two-way binding

02 Design reativo

computed · resources · effects · auto-tracking · glitch-free · imutabilidade

03 Signal graph + exercício

pensar no grafo · fluxo com função pura · Boarding List (consolidação)



01

Sub-componentes (cont.)

API pública, input()/output(), content projection, model() e two-way binding

Começar pelo pai: a API pública do FlightCard

flight-search.html – como o pai o vai usar

```
<app-flight-card
  [item]="flight"           ← propriedade (in)
  [selected]="basket()[flight.id]" ← propriedade (in)
  (selectedChange)="updateBasket(flight.id, $event)" ← evento (out)
/>
```

flight-search.ts

```
// vários voos selecionados: id → boolean
protected readonly basket =
  signal<Record<number, boolean>>({ 3: true, 5: true });

protected updateBasket(flightId: number, selected: boolean): void {
  this.basket.update(b => ({ ...b, [flightId]: selected }));
  // ↑ nova referência
}
```

Porquê o spread?

- Novo objeto → Angular deteta a mudança
- Mutar `b[id] = true` **não** dispara nada

input() e output(): signals de entrada, eventos de saída

```
ui/flight-card/flight-card.ts
import { input, output } from '@angular/core';

export class FlightCard {
  readonly item = input.required<Flight>(); // InputSignal<Flight>
  readonly selected = input(false); // InputSignal<boolean>
  readonly selectedChange = output<boolean>(); // OutputEmitterRef

  protected select() { this.selectedChange.emit(true); }
  protected deselect() { this.selectedChange.emit(false); }
}
```

Regras

- `required` → sem default; falta = erro de build
- Inputs são signals **read-only**
- Para cima só por evento: `.emit()`
- `readonly` protege a referência, não o emit

Typo clássico: (`selectedChange`) — compila, nunca dispara. Verificar o nome do evento antes de culpar o `emit()`.

Template do card, @let e ligação no pai

```
flight-card.html
@let itemValue = item();
<div [class.selected]="selected()" class="card">
  <h2>{{ itemValue.from }} - {{ itemValue.to }}</h2>
  <p>#{{ itemValue.id }} · {{ itemValue.date | date }}</p>
  @if (!selected()) {
    <button (click)="select()">Select</button>
  } @else {
    <button (click)="deselect()">Remove</button>
  }
</div>
```

```
pai: importar + usar
@Component({ imports: [FormField, FlightCard, JsonPipe] })

<!-- flight-search.html -->
@for (flight of flights(); track flight.id) {
  <app-flight-card
    [item]="flight"
    [selected]="basket() [flight.id]"
    (selectedChange)="updateBasket(flight.id, $event)"
  />
}
<pre>{{ basket() | json }}</pre>
```

- @let = variável local do template; poupa chamadas repetidas ao getter
- \$event dentro de um event binding é o valor passado ao emit()
- Fluxo: clique → emit → updateBasket → basket.update → [selected] volta a descer

Content projection: o pai decide parte do markup

```
pai — conteúdo projetado
<app-flight-card [item]="flight" ...>
  <div>O voo #{ flight.id } pode atrasar. Consulte os monitores.</div>
</app-flight-card>
```

```
filho — 1 slot
<!-- um único slot, com conteúdo por defeito -->
<ng-content>
  <p>Sem informação adicional.</p>
</ng-content>
```

```
filho — N slots
<!-- vários slots por seletor CSS -->
<ng-content select=".detail" />
<ng-content select=".button" />
<ng-content select=".footer">Reservados.</ng-content>
```

- Personalizar o filho **sem** o alterar — ideal para componentes de UI partilhados
- `select` aceita qualquer seletor CSS (classe, tag, atributo)
- Conteúdo projetado é avaliado no contexto do **pai** (`flight.id` acima é do pai)

model(): um input que também escreve

```
flight-card.ts - versão com model()
import { input, model } from '@angular/core';

export class FlightCard {
  readonly item = input.required<Flight>();
  readonly selected = model(false); // ModelSignal<boolean>

  protected select() { this.selected.set(true); }
  protected deselect() { this.selected.set(false); }
}
```

O pai não muda

- [selected] continua a existir
- (selectedChange) é gerado automaticamente
- Regra: propriedade + Change

model()

Filho escreve de volta **de imediato** — menos código, ideal para controlos simples

input() + output()

Filho decide **quando** notificar (validar, agrupar, adiar) — controlo fino

Two-way binding: [(value)] é só açúcar

```
delay-stepper.ts
export class DelayStepper {
  readonly value = model(0);
  increase() { this.value.update(v => v + 15); }
  decrease() { this.value.update(v => Math.max(v - 15, 0)); }
}
```

```
mesma API pública, sem model()
// equivalente explícito
readonly value = input.required<number>();
readonly valueChange = output<number>();
increase() { this.valueChange.emit(this.value() + 15); }
```

```
pai (flight-search)
protected readonly maxDelay = signal(0);

<!-- forma longa -->
<app-delay-stepper [value]="maxDelay()"
                  (valueChange)="maxDelay.set($event)" />

<!-- forma curta: sem chamar o getter! -->
<app-delay-stepper [(value)]="maxDelay" />
```

Condições

- Propriedade `value`
- Evento `valueChange` que emite o **novo valor**
- É tudo — o resto é convenção

Relembrar: o que acabámos de escrever

FlightCard, cesto de voos e DelayStepper

O que ficou no código

- FlightCard em ui/ com item (required) e selected
- selectedChange → updateBasket no pai (spread!)
- Tabela substituída por @for de cards, @let no template
- Aviso de atraso projetado via <ng-content>
- DelayStepper com model() e [(value)]="maxDelay"

Para explorar

- Passar o card para model(false) — o pai muda? (não)
- Slot extra select=".footer" com conteúdo por defeito
- Provocar o typo (seletedChange) e ver que não há erro

Fluxo: clique → emit → updateBasket → basket.update (nova referência) → [selected] volta a descer.

Ponto de situação: o capítulo dos componentes

Do componente estático ao componente composto — e a ponte para o design reativo

Componente com Signals

- Estado em `signal()`; leitura no template via `getter`
- Signal Forms: `form()` + `[formField]`
- `@if / @for / @switch` + pipes
- Bindings: `(evento) · [prop] · {{ }} · [(x)]`

Acesso a dados

- `HttpClient`: imperativo, `Observable`, todos os verbos
- `httpResource`: declarativo, entra `signal` / sai `signal`
- `value · error · isLoading · status`
- Race conditions tratadas; `reload()` = `exhaustMap`

Sub-componentes

- `input()` / `input.required()` → `read-only`
- `output()` + `emit()` para cima
- `model()` = `input` + evento `Change`
- `<ng-content select>` para o pai personalizar

Fio condutor: `signals` + `OnPush` funcionam porque os dados mudam por **nova referência** (`spread`). A seguir: `computed`, `resources` e `effects` — o design reativo.

Checkpoint: `git pull` → tag `sessao-03a` (fim do capítulo dos componentes)



02

Design reativo com Signals

Deixar de descrever COMO atualizar; descrever DE QUE depende cada valor

Pensar como numa folha de cálculo

Não escrevo "quando A mudar, atualiza B". Escrevo " $B = f(A)$ " e o framework mantém tudo sincronizado.

`computed()`

- Derivar **síncrono**
- Lazy: só recalcula quando é lido
- Read-only
- 1.^a escolha, quase sempre

`resource`

- Derivar **assíncrono** (HTTP, Promise, Observable)
- Entrada e saída são signals
- Trata race conditions

`effect()`

- Reagir com um **side effect**
- Não devolve valor
- Para "a última milha": canvas, toast, log, libs externas
- Usar com moderação

Preferência: `computed` → `resource` → `effect`. Se um `effect` escreve em signals, provavelmente devia ser um `computed`.

computed() e untracked()

derivar

```
import { computed, signal } from '@angular/core';

protected readonly filter = signal({ from: 'Lisbon', to: 'Porto' });

// recalcula quando filter muda — e só quando alguém lê
protected readonly flightRoute = computed(
  () => `${this.filter().from} → ${this.filter().to}`,
);

<h2>Rota: {{ flightRoute() }}</h2>
```

ler sem seguir

```
import { untracked } from '@angular/core';

protected readonly from = computed(() => this.filter().from);
protected readonly to   = computed(() => this.filter().to);

protected readonly flightRoute = computed(() => {
  const origin = this.from(); // tracked
  const dest = untracked(() => this.to()); // NÃO tracked
  return `${origin} → ${dest}`;
}); // só reage a 'from'
```

- `computed` devolve `Signal<T>` (read-only) — usa-se no template como qualquer signal
- **Lazy + memoizado**: sem leitores, não há cálculo; sem mudança nas dependências, não há recálculo
- `untracked()` é uma ferramenta de precisão — se precisa dela muito, repensar o desenho

Três resources, uma API para o consumidor

	httpResource	rxResource	resource
Import	@angular/common/http	@angular/core/rxjs-interop	@angular/core
Entrada	request: () => { url, params }	params: () => {...}	params: () => {...}
Loader	(interno: HttpClient)	stream: (p) => Observable	loader: (p) => Promise
Quando	GET direto a uma Web API	Serviços com Observables · compor com RxJS	Raro — base dos outros dois
Cancelamento	automático	automático (unsubscribe)	via AbortSignal no loader

Comum a todos: value · error · isLoading · status · reload() · devolver undefined nos params desativa o resource · só a última resposta conta.

rxResource: quando já existe um Observable

```
flight-search.ts
import { rxResource } from '@angular/core/rxjs-interop';

private readonly http = inject(HttpClient);

protected readonly flightsResource = rxResource({
  params: () => ({ ...this.filter() }), // computed por baixo
  stream: ({ params }) => this.find(params.from, params.to),
  defaultValue: [],
});

private find(from: string, to: string): Observable<Flight[]> {
  return this.http.get<Flight[]>(API_URL, { params: { from, to } });
}
```

`params` recebe valores simples, não signals: o resource já leu os signals por nós.

rxjs-interop

- `rxResource` — `Observable` → `resource`
- `toSignal(obs$)` — `Observable` → `Signal`
- `toObservable(sig)` — `Signal` → `Observable`
- A ponte entre os dois mundos (módulo 5)

resource: a base, baseada em Promises

Promise-based

```
import { resource } from '@angular/core';

protected readonly flightsResource = resource({
  params: () => ({ from: this.filter().from, to: this.filter().to }),
  loader: ({ params, abortSignal }) =>
    this.findPromise(params.from, params.to, abortSignal),
  defaultValue: [],
});
```

- `httpResource` e `rxResource` são abstrações por cima deste
- Útil com `fetch()` nativo ou SDKs que devolvem Promises
- Se já tem um Observable, **não converta** — use `rxResource`

AbortSignal

- API do **browser** para cancelar operações assíncronas
- Nada a ver com Angular Signals
- Sem ele, o resource apenas **ignora** respostas antigas

effect(): reagir com um side effect

```
flight-search.ts
import { effect } from '@angular/core';

constructor() {
  // corre 1x, e depois sempre que filter mudar
  effect(() => {
    const f = this.filter();
    console.log('From:', f.from, 'To:', f.to);
  });
}

// bom uso: algo que o data binding não consegue fazer
effect(() => {
  const error = this.error();
  if (error) this.toast.show('Erro a carregar voos');
});
```

Onde

- Precisa de **injection context**: constructor ou inicializador de campo
- Cabos instalam-se quando a casa se constrói

Para quê

- "Última milha": canvas, toast, log, storage, libs sem signals
- **Não** para escrever noutros signals nem para lógica de negócio

Effects que precisam do DOM pronto

Função	Corre quando	Reativo a signals?	Uso típico
effect()	após mudança de signal (antes do render)	sim	log, toast, storage
afterRenderEffect()	após mudança de signal, depois do render	sim	medir/alterar DOM em função de estado
afterNextRender()	1× após o próximo render	não	inicializar lib de charts, scroll inicial
afterEveryRender()	após cada render	não	sincronizar com lib externa a cada ciclo

```
afterRenderEffect(() => {  
  const el = this.chartHost().nativeElement; // DOM já renderizado  
  drawChart(el, this.flightsWithDelays()); // reage a mudanças  
});
```


Signals no ciclo de vida: inputs e o constructor

```
flight-card.ts
export class FlightCard {
  readonly item = input.required<Flight>();

  // ✅ computed é lazy: só lê item() quando alguém lê flightRoute()
  protected readonly flightRoute = computed(
    () => `${this.item().from} → ${this.item().to}`,
  );

  constructor() {
    // console.log(this.item()); ❌ input não inicializado
    effect(() => console.log('Item:', this.item())); // ✅ adiado
  }
}
```

Regra

- Constructor = instânciação; o binding inicial vem **depois**
- `effect` no constructor é adiado até à inicialização
- `computed` só lê quando é lido
- Adeus `ngOnChanges` para derivar de inputs

Tempo: `debounced()` (Angular 22)

```
debounce de um signal
import { debounced, effect, signal } from '@angular/core';

const filter = signal('');
const debouncedFilter = debounced(filter, 300); // → Resource

effect(() => console.log(debouncedFilter.value()));
// filter muda a cada tecla; value() só após 300 ms estável
// debouncedFilter.status() === 'loading' na janela
```

Porquê um Resource?

- Signals **não têm tempo**; resources sim
- `value lag` + `status` de espera

Armadilha: `debounced()` devolve um Resource read-only — não pode substituir o signal escrevível que o form usa. Manter os dois: o original para o form, o `debounced` para a pesquisa. Para `debounce` por campo, Signal Forms traz `debounce()` (módulo 7).

Auto-tracking: o que é seguido, e onde

tracking atravessa chamadas

```
constructor() {  
  effect(() => {  
    this.logCriteria(); // segue filter, sem o mostrar  
  });  
}  
  
private logCriteria(): void {  
  const f = this.filter(); // tracked, indiretamente  
  console.log(f.from, '→', f.to);  
}
```

lógica de negócio num effect

```
effect(() => {  
  const criteria = this.criteria();  
  this.service.executeLogic(criteria); // ✗  
});  
  
// se executeLogic lê isLoading() ou userId()...  
// ...o effect re-corre quando ELES mudam.  
// e não há tratamento de race conditions.
```

Contexto reativo existe em

Templates e effects. `computed` herda o contexto de quem o lê.

Consequência

Effects para rendering: ótimo. Effects para negócio: re-execuições surpresa + sem race handling.

Explicit effects e untracking condicional

"explicit effect" — padrão controverso

```
// ✅ só criteria dispara — mas esconde dependências
effect(() => {
  const criteria = this.criteria();
  untracked(() => this.service.executeLogic(criteria));
});
```

dependências dinâmicas

```
effect(() => {
  if (isDelayed()) {
    console.log(delay()); // só seguido se isDelayed
  }
});

effect(() => {
  const isDelayedValue = isDelayed();
  const delayValue = delay(); // ler no início
  if (isDelayedValue) console.log(delayValue);
});
```

- Um signal que **deixa de ser lido** numa execução deixa de ser seguido — sem leaks, mas com surpresas
- Se quer que um signal dispare sempre, leia-o no início do `effect` / `computed`
- Vale para templates também: um `@if` fechado deixa de seguir o que está lá dentro

Glitch-free: nunca ver estados intermédios

5 updates, 1 execução

```
constructor() {  
  effect(() => {  
    const f = this.filter();  
    console.log('from:', f.from, 'to:', f.to);  
  });  
  
  setTimeout(() => {  
    this.filter.update(f => ({ ...f, from: 'Paris' }));  
    this.filter.update(f => ({ ...f, from: 'Frankfurt' }));  
    this.filter.update(f => ({ ...f, from: 'New York' }));  
    this.filter.update(f => ({ ...f, to: 'Berlin' }));  
    this.filter.update(f => ({ ...f, to: 'London' }));  
  }, 2000);  
}  
// consola: 'from: New York to: London' — UMA vez
```

Vantagem

- Consistência garantida
- Zero renders intermédios

Consequência

- Signals **não** representam eventos nem streams no tempo
- Mensagens rápidas perdem-se → RxJS

Igualdade e imutabilidade

```
=== e spread
const count = signal(0);
count.set(0); count.set(0); // === igual → nada acontece

// objetos: === compara REFERÊNCIA
flight.update(f => ({ ...f, date: newDate, delayed: true }));
// shallow copy + override → nova instância → detetada

// ✗ mutação in-place: mesma referência, ninguém repara
flight().delayed = true;
```

E o OnPush?

- No @for, cada flight é comparado por ===
- Objeto novo → só **esse** FlightCard é revisitado
- Imutabilidade é o que torna OnPush barato

Regra curta: ao atualizar objetos ou arrays ligados a bindings, crie sempre novas instâncias. Custom equality (2.º arg. de signal()) existe, mas raramente compensa.

Pensar no grafo de signals



Implementar o fluxo: computed + função pura

flight-search.ts

```
protected readonly flights = this.flightsResource.value;
protected readonly delayInMin = signal(0);

protected readonly flightsWithDelays = computed(() =>
  toFlightsWithDelays(this.flights(), this.delayInMin()),
);

protected delay(): void {
  this.delayInMin.update(d => d + 15); // só isto
}
```

helper puro

```
// fora da classe: pura, sem signals
function toFlightsWithDelays(
  flights: Flight[], delay: number): Flight[] {
  if (flights.length === 0) return [];
  const MIN = 60_000;
  const [first, ...rest] = flights;
  const t = new Date(first.date).getTime();
  const newDate = new Date(t + delay * MIN);
  const newFirst = { ...first, date: newDate.toISOString() };
  return [newFirst, ...rest]; // novo array
}
```

```
<button type="button" (click)="delay()" [disabled]="flights().length === 0">Delay</button>
@for (flight of flightsWithDelays(); track flight.id) { <app-flight-card [item]="flight" ... /> }
```

Funções puras fora da classe: sem acesso a `this`, e obrigam a ler os signals dentro do `computed` — as dependências ficam visíveis à primeira.

Relembrar: o que acabámos de escrever

O fluxo reativo no Skyboard

O que ficou no código

- `flightRoute` com `computed` a partir do `filter`
- `delayInMin` + `flightsWithDelays` (`computed` + `helper` puro)
- Botão `Delay`; `@for` sobre `flightsWithDelays()`
- `effect` no constructor a fazer `log` do `status()`

Para explorar

- Ligar o `DelayStepper` (`[(value)]`) ao `delayInMin`
- Provar o glitch-free: 3 `update` seguidos, 1 `log`
- Trocar `httpResource` por `rxResource` sem mudar o `template`

Pergunta para discutir: onde é que, no vosso código atual, um `ngOnChanges` ou um `subscribe` podia ser um `computed`?

Exercício — Boarding List

Consolidação: signals + computed · níveis progressivos · sem HTTP

```
setup
ng g c domains/ticketing/
    feature-boarding/boarding-list

export interface Passenger {
  id: number;
  name: string;
  boarded: boolean;
}
```

Nível 1 — Lista (≈15 min)

- `passengers = signal<Passenger[]>([...])` com 3 iniciais
- `newName = signal('') + form() + [formField]`
- `add(): update` com **array novo**; depois `newName.set('')`
- `toggle(id): map + spread` → **objeto novo**
- `@for ... track p.id + @empty + checkbox`

Nível 2 — Lote e contagem (≈10 min)

- "Embarcar todos" e "Reset" com `update + map`
- `boardedCount` e `total` com `computed()`; `{{ boardedCount() }}` / `{{ total() }}`
- `allBoarded = computed(...)` encadeado → `@if "Porta fechada — bom voo!"`

Perguntas para discutir

- Porque é que `passengers().push(novo)` não atualiza o ecrã?
- Porque é que `p.boarded = !p.boarded` no `map` é má ideia?
- `boardedCount` recalcula ao escrever no input `newName`? Porquê não?

Exercício — Boarding List (cont.)

Nível 3 e 4 · o que sobrar fica para casa

Nível 3 — Filtro (≈10 min)

- `filter = signal<'all' | 'boarded' | 'pending'>('all')`
- **3 botões** com `filter.set(...)` e `[class.active]`
- `visible = computed(...)` que filtra por `filter()`
- `@for` passa a iterar `visible()`
- `@switch (filter())` para o título, com `@default never`

```
@switch (filter()) {  
  @case ('all') { <h2>Todos</h2> }  
  @case ('boarded') { <h2>Embarcados</h2> }  
  @case ('pending') { <h2>Por embarcar</h2> }  
  @default never;  
}
```

Nível 4 — Sub-componente (bônus)

- `ng g c domains/ticketing/ui/passenger-row`
- `passenger = input.required<Passenger>()`
- `boardedChange = output<boolean>()`
- `<app-passenger-row [passenger]="p" (boardedChange)="toggle(p.id)" />`
- Refatorar para `model(false)` — dá para `[(boarded)]`?

Pergunta final: o que acontece se adicionarem `'crew'` ao union type sem `@case`? Experimentem — e guardem a resposta para a próxima sessão.

Resumo da sessão

Acesso a dados

- `HttpClient` para escrever
- `httpResource` para ler
- `value · error · isLoading · status`
- Race conditions tratadas

Sub-componentes

- `input()` / `input.required()`
- `output()` + `emit()`
- `model()` = `input` + `Change`
- `[(x)]` = açúcar
- `<ng-content select>`

Design reativo

- `computed` > `resource` > `effect`
- Auto-tracking · untracked
- Glitch-free · imutabilidade
- Pensar no grafo

Para casa: Boarding List (níveis que faltarem). Próxima sessão: Router — navegação, rotas parametrizadas, lazy loading.
Antes: git pull → tag sessao-03.